APIs & Web Services I

REST Principles • CRUD Operations • JSON Exchange • Flask

Session 5 of 7 | 4 Hours

Big Data Engineering Programme
Academic Year 2025–2026

Session Overview

Duration	4 hours (theory + lab)
Theory	REST principles, HTTP verbs, status codes, JSON, URL design
Lab	Build a Flask REST API exposing Kafka-streamed sensor data
Prerequisites	Sessions 1–4, Python fundamentals
Tools used	Python 3, Flask, kafka-python-ng, curl / Postman

Learning Objectives

By the end of this session, students will be able to:

- Define REST and explain its six architectural constraints.
- Map CRUD operations to the correct HTTP verbs and design resource-oriented URLs.
- Read and write well-formed JSON payloads and interpret HTTP status codes.
- Build a Flask REST API with multiple endpoints that serve data from Parquet and Kafka.
- Test API endpoints with `curl` and Postman.
- Handle errors gracefully and return appropriate HTTP status codes.

Contents

1. Part I – Web Services and APIs

1.1 What Is a Web Service?

Definition – Web Service

A **web service** is a software component that exposes functionality or data over a *network* using standardised protocols (typically HTTP/HTTPS), allowing applications written in any language on any platform to communicate with each other.

In the context of data engineering, web services serve a critical role: they are the **serving layer** that makes processed data accessible to consumers – dashboards, mobile apps, ML inference services, and other pipelines.

1.2 API vs Web Service

- An **API** (Application Programming Interface) is any interface through which software components communicate. This includes library APIs, OS APIs, and network APIs.
- A **Web Service API** specifically uses HTTP/HTTPS as the transport. All web services expose an API, but not all APIs are web services.

1.3 Evolution of Web Service Styles

Table 1: Web service architectural styles compared

lightbg Style	Protocol / Format	Strengths	Era
SOAP	XML over HTTP/SMTP; WSDL contract	Strict contracts, enterprise tooling	2000s
REST	HTTP + JSON/XML; no formal contract	Simple, stateless, cacheable, ubiquitous	2005–present
GraphQL	HTTP + custom query language	Client controls shape of response	2015–present
gRPC	HTTP/2 + Protobuf binary	High performance, streaming, typed	2016–present

Why REST for This Course?

REST is the **dominant style for data APIs** because it: uses standard HTTP (every language has an HTTP client), is human-readable with JSON, is stateless (easy to scale horizontally), and is well-understood by every developer and data consumer.

2. Part II – REST: Principles and Constraints

2.1 Defining REST

Definition – REST

REST (Representational State Transfer) is an architectural *style* (not a protocol or standard) for distributed hypermedia systems, defined by Roy Fielding in his 2000 doctoral dissertation. A service that adheres to REST's constraints is called *RESTful*.

2.2 The Six REST Constraints

Fielding defined six constraints that a system must satisfy to be RESTful:

1. **Client–Server Separation**
   The client (UI, consumer) and server (API, data) are decoupled. They interact only through the API interface. Either side can evolve independently as long as the interface contract is preserved.

2. **Statelessness**
   Each request from a client must contain *all* information needed to process it. The server stores no session state between requests. Authentication credentials, filters, and pagination parameters must be sent with every request.

   Statelessness Benefits

   - Any server instance can handle any request (horizontal scaling without sticky sessions).
   - Failed requests can be retried without side effects.
   - No server-side session cleanup needed.

3. **Cacheability**
   Responses must be labelled as *cacheable* or *non-cacheable*. Cacheable responses can be reused by the client or an intermediate proxy, reducing load and latency. HTTP headers `Cache-Control`, `ETag`, and `Last-Modified` implement this.

4. **Uniform Interface**
   The central REST constraint. It has four sub-constraints:

   - **Resource identification in requests**: resources are identified by URIs (e.g., `/sensors/temperature`).
   - **Manipulation through representations**: clients interact with resources via representations (JSON, XML), not directly with the resource.
   - **Self-descriptive messages**: each message includes enough information to describe how to process it (Content-Type header).

- **HATEOAS**: Hypermedia As The Engine Of Application State – responses include links to related actions (advanced; not always implemented).

5. **Layered System**
   The client cannot tell whether it is connected directly to the server or to an intermediary (load balancer, API gateway, cache). Layers may be added transparently.

6. **Code on Demand** (optional)
   Servers may send executable code to clients (e.g., JavaScript). The only optional constraint; rarely used in data APIs.

2.3 Resources and Representations

Definition – Resource

A **resource** is any concept that can be named and addressed: a sensor reading, a list of alerts, a user profile, an aggregated daily report. Resources are identified by **URIs** (Uniform Resource Identifiers).

Definition – Representation

A **representation** is the current state of a resource encoded in a specific format (JSON, XML, CSV). The client requests a specific representation via the `Accept` header; the server declares the format it returns via `Content-Type`.

3. Part III – HTTP: Verbs, Status Codes, and Headers

3.1 HTTP Request Structure

Every HTTP request has four parts:

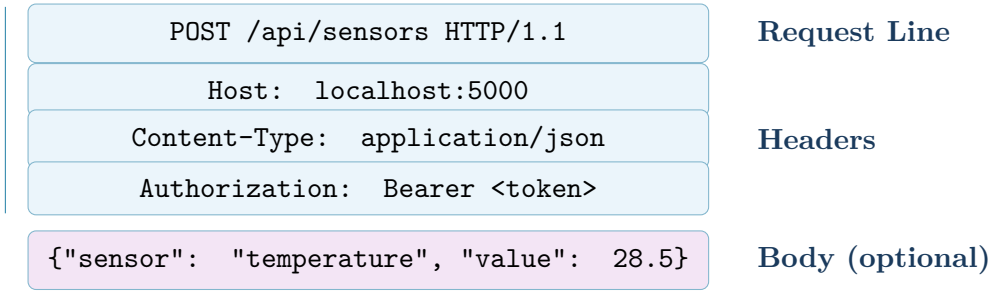

Figure 1: HTTP request structure: request line, headers, and body.

3.2 HTTP Verbs and CRUD Mapping

REST maps the four CRUD operations to HTTP verbs. Each verb has specific semantics that must be respected:

Table 2: HTTP verbs, CRUD operations, and semantics

lightbg **Verb**	**CRUD**	**Semantics**	**Idempotent?**	**Safe?**
GET	Read	Retrieve a resource or collection. Never modifies state.	Yes	Yes
POST	Create	Create a new resource. Returns the created resource + 201.	No	No
PUT	Update (replace)	Replace the entire resource at the given URI.	Yes	No
PATCH	Update (partial)	Apply partial modifications to a resource.	No	No
DELETE	Delete	Remove the resource at the given URI.	Yes	No

Idempotent vs Safe

- **Safe**: the operation has no side effects on the server (read-only). GET and HEAD are safe.
- **Idempotent**: calling the same operation N times produces the same result as calling it once. GET, PUT, and DELETE are idempotent. POST is not (calling POST twice creates two resources).

These properties are critical for retry logic in distributed systems.

3.3 HTTP Status Codes

Status codes are grouped into five classes:

Table 3: HTTP status code families

lightbg **Range**	**Category**	**Meaning**
2xx	**Success**	Request received, understood, accepted
3xx	**Redirection**	Further action needed to complete request
4xx	**Client Error**	Request has bad syntax or cannot be fulfilled
5xx	**Server Error**	Server failed to fulfil a valid request

Table 4: Essential status codes for data APIs

`lightbg Code`	**Name**	**When to use**
200	OK	Successful GET, PUT, PATCH, DELETE
201	Created	Successful POST; include `Location` header
204	No Content	Successful DELETE with no body
400	Bad Request	Malformed JSON, missing required field
401	Unauthorized	Authentication required or token missing
403	Forbidden	Authenticated but not authorised
404	Not Found	Resource does not exist
409	Conflict	Resource already exists (duplicate)
422	Unprocessable Entity	Valid JSON but business validation failed
500	Internal Server Error	Unhandled exception on the server
503	Service Unavailable	Server temporarily overloaded or down

3.4 Key HTTP Headers

Table 5: Important HTTP headers for REST APIs

`lightbg Header`	**Direction**	**Purpose**
`Content-Type`	Request / Response	Media type of the body (e.g., `application/json`)
`Accept`	Request	Media types the client can process
`Authorization`	Request	Credentials (`Bearer <token>`, `Basic <b64>`)
`Location`	Response	URI of newly created resource (201 responses)
`X-Request-ID`	Request / Response	Unique ID for request tracing across services
`Cache-Control`	Response	Caching directives (`no-cache`, `max-age=3600`)

4. Part IV – URL Design and JSON

4.1 Designing Resource-Oriented URLs

Good URL design is one of the most visible aspects of API quality. Here are the core rules:

4.1.1 Rule 1: Use Nouns, Not Verbs

The HTTP verb already expresses the action. The URL should identify the *resource*, not the operation.

lightbg **Pattern**	**Bad**	**Good**
Get sensors	`GET /getSensors`	`GET /sensors`
Create alert	`POST /createAlert`	`POST /alerts`
Delete reading	`GET /deleteReading?id=5`	`DELETE /readings/5`

4.1.2 Rule 2: Use Plural Nouns for Collections

```
GET  /sensors                  # collection of all sensors
GET  /sensors/temperature      # specific sensor type
GET  /sensors/temperature/readings     # readings collection
GET  /sensors/temperature/readings/42 # specific reading
```

4.1.3 Rule 3: Use Hierarchical Nesting for Relationships

```
GET /sensors/{type}/readings           # readings for a sensor
GET /sensors/{type}/readings/latest    # latest reading
GET /sensors/{type}/alerts             # alerts for a sensor
```

4.1.4 Rule 4: Use Query Parameters for Filtering

Path parameters identify a specific resource; query parameters filter or paginate a collection.

```
# Filter by date range
GET /readings?from=2024-01-15&to=2024-01-16

# Pagination
GET /readings?page=2&per_page=50

# Combination
GET /sensors/temperature/readings?from=2024-01-15&anomaly=true
```

4.1.5 Rule 5: Version Your API

Always include a version prefix to allow breaking changes without disrupting existing consumers:

```
GET /api/v1/sensors
GET /api/v2/sensors    # new version, old still works
```

4.2 JSON Data Exchange

Definition – JSON

JSON (JavaScript Object Notation) is a lightweight, human-readable data-interchange format. It is the de facto standard payload format for REST APIs because it is natively supported by every programming language and all modern HTTP clients.

4.2.1 JSON Data Types

```
{
  "string":   "hello world",
  "number":   42.5,
  "boolean":  true,
  "null":     null,
  "array":    [1, 2, 3],
  "object":   { "nested_key": "nested_value" }
}
```

4.2.2 Standard API Response Envelopes

A consistent response structure makes APIs predictable:

```
// Success response (single resource)
{
  "status":  "success",
  "data": {
    "sensor_type": "temperature",
    "value":       28.5,
    "unit":        "C",
    "timestamp":   "2024-01-15T10:23:45Z"
  }
}

// Success response (collection)
{
  "status":  "success",
  "count":   3,
  "data": [
    { "sensor_type": "temperature", "value": 28.5 },
    { "sensor_type": "temperature", "value": 31.2 }
  ]
```

```
}

// Error response
{
  "status":  "error",
  "code":    404,
  "message": "Sensor type 'radar' not found.",
  "errors":  []
}
```

Common Pitfall

Never expose internal error details in API responses. A stack trace or database error message returned to the client is a security vulnerability (information disclosure). Always return a friendly, generic error message and log the full exception server-side.

5. Part V – Flask: Building REST APIs in Python

5.1 What Is Flask?

Definition – Flask

Flask is a **lightweight Python web framework** (a "micro-framework") that provides the minimal tools needed to build web applications and REST APIs: URL routing, request parsing, and response building. Unlike Django, Flask imposes no structure – you add only what you need.

Flask is the most popular Python framework for data API prototyping because of its simplicity, flexibility, and native JSON support.

5.2 Flask Core Concepts

lightbg Concept	Description
`Flask(__name__)`	Creates the application instance. `__name__` tells Flask where to look for templates and static files.
`@app.route("/path")`	Decorator that maps a URL path to a Python function (the *view function*).
`methods=["GET","POST"]`	List of HTTP verbs the route accepts. Default: GET only.
`request`	Global proxy object containing the incoming HTTP request (headers, body, query parameters).
`jsonify(data)`	Converts a Python dict/list to a JSON HTTP response with `Content-Type: application/json`.
`abort(code)`	Immediately return an HTTP error response (e.g., `abort(404)`).
`Blueprint`	Group related routes into a reusable module.

5.3 Minimal Flask Application

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/api/v1/health")
def health_check():
    return jsonify({"status": "ok", "version": "1.0"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

@app.route("/api/v1/health"): this decorator registers the function `health_check()` as the handler for `GET /api/v1/health`.
jsonify(...): Flask serialises the dict to JSON and sets the `Content-Type: application/json` header automatically.
debug=True: enables the auto-reloader (restart on code change) and the interactive debugger. **Never use in production**.

6. Part VI – Lab Session

Lab 5 – Flask REST API Exposing Sensor Data

Duration	90 minutes
Objective	Build a Flask REST API with endpoints that serve live Kafka data and historical Par
Tools	Python 3, Flask, kafka-python-ng, PySpark, curl
Difficulty	●●● Intermediate

Lab Timeline

Time	Activity	Goal
0–10 min	Setup: install Flask, verify lake data	Environment ready
10–25 min	Step 1: project structure + health check	First endpoint live
25–40 min	Step 2: sensor list and latest reading	GET endpoints
40–55 min	Step 3: statistics from Parquet	Query data lake
55–70 min	Step 4: POST – write to Kafka	Full pipeline
70–80 min	Step 5: error handling	Robust responses
80–90 min	Step 6: test with curl	End-to-end validation

6.1 Step 0 – Setup

6.1.1 0a – Install Dependencies

```
source venv/bin/activate
pip install flask kafka-python-ng pyspark==3.5.3
```

We add `flask` to the existing TP environment. Flask is a pure Python package with no system dependencies. The API will run as a development server on port 5000.

6.1.2 0b – Verify Data Lake Has Data

```
# Check curated Parquet files exist (from TP4)
ls /tmp/datalake/curated/domain=iot/

# If empty, run TP4 pipeline first:
# spark-submit --packages ... datalake_pipeline.py
# python producer.py --count 500
```

Common Pitfall

Python 3.12+ compatibility fixes in helper modules:

1. **`kafka_utils.py` – sort crash on `None` timestamp**
   When `sort(key=lambda r: r.get("timestamp", 0))` is used and a record has `"timestamp": null` in Kafka, Python raises `TypeError: '<' not`

supported between 'NoneType' and 'int'.
Fix: use `r.get("timestamp") or 0` instead.

2. **`lake_utils.py` – PySpark startup crash when Java is absent**
If Java is not installed, PySpark raises a `PySparkRuntimeError` (not an `ImportError`). Catching only `ImportError` lets the exception propagate and returns a 500 to the client.
Fix: catch `Exception` broadly in `_get_spark()`, and move `spark = _get_spark()` *inside* the `try` block in `get_statistics()`. The endpoint then returns an empty list gracefully instead of crashing.

6.1.3 0c – Project Structure

```
mkdir -p sensor_api/
# We will create these files:
# sensor_api/app.py          (Flask application)
# sensor_api/kafka_utils.py (Kafka consumer helper)
# sensor_api/lake_utils.py   (Parquet query helper)
```

6.2 Step 1 – Application Factory and Health Check

6.2.1 1a – Create `app.py`

```
"""
Lab 5 -- Sensor Data REST API
==============================
app.py  -  Flask application entry point
"""
from flask import Flask, jsonify, request, abort
from datetime import datetime, timezone

from kafka_utils import get_latest_readings, publish_reading
from lake_utils  import get_statistics, get_sensor_types

app = Flask(__name__)

#         API Metadata

API_VERSION = "1.0"
API_PREFIX  = "/api/v1"
```

`API_PREFIX = "/api/v1"`: defining the version prefix as a constant means changing it only requires editing one line. All route definitions use this prefix, ensuring consistency. We import helpers from two separate modules (`kafka_utils` and `lake_utils`) to keep `app.py` focused on routing logic only – a clean separation of concerns.

6.2.2 1b – Health Check Endpoint

```python
@app.route(f"{API_PREFIX}/health")
def health():
    """
    GET /api/v1/health
    Returns API health status and metadata.
    Always returns 200 if the server is running.
    Used by load balancers and monitoring systems.
    """
    return jsonify({
        "status":    "ok",
        "version":   API_VERSION,
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "service":   "sensor-data-api",
    }), 200
```

A **health check endpoint** is mandatory in any production API. Load balancers (NGINX, AWS ALB) call this endpoint every few seconds to decide whether to route traffic to this instance. If it returns a non-2xx status, the instance is removed from the pool. **datetime.now(timezone.utc).isoformat()** returns an ISO 8601 timestamp with timezone info: `2024-01-15T10:23:45+00:00`. Always use UTC in API responses – never local time.

6.3 Step 2 – Sensor List and Latest Reading Endpoints

6.3.1 2a – List All Sensor Types

```python
@app.route(f"{API_PREFIX}/sensors")
def list_sensors():
    """
    GET /api/v1/sensors
    Returns the list of all known sensor types.
    Data is read from the Parquet curated zone.
    """
    try:
        sensor_types = get_sensor_types()
        return jsonify({
            "status": "success",
            "count":  len(sensor_types),
            "data":   sensor_types,
        }), 200
    except Exception as exc:
        # Log full error server-side, return generic message
        app.logger.error("list_sensors error: %s", exc)
        return jsonify({
            "status":  "error",
            "message": "Failed to retrieve sensor list.",
        }), 500
```

try/except around every endpoint: unhandled exceptions in Flask return a generic 500 HTML error page. By catching exceptions, we return a structured JSON error response – much more useful for API consumers who are expecting JSON, not HTML.
app.logger.error(...): Flask provides a built-in logger. Always log the full exception (with traceback if needed) server-side so you can debug production issues without exposing details to clients.

6.3.2 2b – Get Latest Reading for a Sensor Type

```
VALID_SENSORS = {"temperature", "humidity", "pressure"}

@app.route(f"{API_PREFIX}/sensors/<sensor_type>/latest")
def latest_reading(sensor_type: str):
    """
    GET /api/v1/sensors/{sensor_type}/latest
    Returns the most recent reading from Kafka
    for the given sensor type.

    Path parameters:
      sensor_type (str): one of temperature, humidity, pressure

    Returns:
      200 + reading if found
      404 if sensor_type is unknown
    """
    #         Validate path parameter

    if sensor_type not in VALID_SENSORS:
        return jsonify({
            "status":  "error",
            "message": (
                f"Unknown sensor type '{sensor_type}'. "
                f"Valid types: {sorted(VALID_SENSORS)}"
            ),
        }), 404

    try:
        reading = get_latest_readings(sensor_type, n=1)
        if not reading:
            return jsonify({
                "status":  "error",
                "message": f"No readings available for "
                           f"sensor '{sensor_type}'.",
            }), 404

        return jsonify({
            "status": "success",
            "data":   reading[0],
        }), 200

    except Exception as exc:
```

```
        app.logger.error("latest_reading error: %s", exc)
        return jsonify({
            "status":  "error",
            "message": "Failed to retrieve latest reading.",
        }), 500
```

<sensor_type> in the route is a **URL variable**. Flask extracts the value from the URL and passes it as a function argument. For example, `GET /api/v1/sensors/temperature/latest` calls `latest_reading("temperature")`.
Validate before querying: always validate path parameters against a whitelist before passing them to any backend system. An unvalidated path parameter could be used for injection attacks.
Two distinct 404 cases: (1) the sensor type doesn't exist – a client programming error; (2) no data for a valid sensor – a data availability issue. Both return 404 but with different messages.

6.4 Step 3 – Statistics from the Parquet Data Lake

```
@app.route(f"{API_PREFIX}/sensors/<sensor_type>/stats")
def sensor_stats(sensor_type: str):
    """
    GET /api/v1/sensors/{sensor_type}/stats
    Returns daily statistics for a sensor type.

    Query parameters (optional):
      days (int): number of recent days (default: 7)

    Example:
      GET /api/v1/sensors/temperature/stats?days=3
    """
    if sensor_type not in VALID_SENSORS:
        return jsonify({
            "status":  "error",
            "message": f"Unknown sensor type '{sensor_type}'.",
        }), 404

    #         Parse and validate query parameter

    try:
        days = int(request.args.get("days", 7))
        if days < 1 or days > 90:
            raise ValueError("days must be between 1 and 90")
    except (ValueError, TypeError) as exc:
        return jsonify({
            "status":  "error",
            "message": f"Invalid 'days' parameter: {exc}",
        }), 400

    try:
        stats = get_statistics(sensor_type, days=days)
        return jsonify({
```

```
            "status":      "success",
            "sensor_type": sensor_type,
            "days":        days,
            "count":       len(stats),
            "data":        stats,
        }), 200

    except Exception as exc:
        app.logger.error("sensor_stats error: %s", exc)
        return jsonify({
            "status":  "error",
            "message": "Failed to retrieve statistics.",
        }), 500
```

request.args.get("days", 7): `request.args` is a dictionary-like object containing all query parameters from the URL. The second argument is the default value if the parameter is absent.
Validate query parameters as strictly as path parameters. A user could send `?days=-99` or `?days=abc`. Always validate type (is it an integer?) and range (is it sensible?) before passing to backend systems.
400 vs 422: use 400 (Bad Request) for syntactically malformed input (not an integer). Use 422 (Unprocessable Entity) for syntactically valid but semantically invalid input (e.g., end_date before start_date).

6.5 Step 4 – POST: Write a Reading to Kafka

```
import json

REQUIRED_FIELDS = {"sensor", "value"}

@app.route(f"{API_PREFIX}/readings", methods=["POST"])
def create_reading():
    """
    POST /api/v1/readings
    Publish a new sensor reading to the Kafka topic.

    Request body (JSON):
      {
        "sensor": "temperature",
        "value":  28.5,
        "unit":   "C"       (optional)
      }

    Returns:
      201 + published message metadata on success
      400 if body is malformed or missing required fields
    """
    #         Parse JSON body

    # get_json(silent=True) returns None if body is not valid
       JSON
```

```
    body = request.get_json(silent=True)
    if body is None:
        return jsonify({
            "status":  "error",
            "message": "Request body must be valid JSON. "
                       "Set Content-Type: application/json.",
        }), 400

    #         Validate required fields

    missing = REQUIRED_FIELDS - set(body.keys())
    if missing:
        return jsonify({
            "status":  "error",
            "message": f"Missing required fields: {missing}",
        }), 400

    #         Validate sensor type

    sensor = body["sensor"]
    if sensor not in VALID_SENSORS:
        return jsonify({
            "status":  "error",
            "message": f"Invalid sensor type '{sensor}'.",
        }), 422

    #         Validate value

    try:
        value = float(body["value"])
    except (ValueError, TypeError):
        return jsonify({
            "status":  "error",
            "message": "'value' must be a number.",
        }), 422

    #         Build the message and publish to Kafka

    reading = {
        "sensor":    sensor,
        "value":     value,
        "unit":      body.get("unit", ""),
        "timestamp": int(datetime.now(timezone.utc)
                         .timestamp() * 1000),
        "source":    "api-v1",
        "anomaly":   False,
    }

    try:
        metadata = publish_reading(reading)
        return jsonify({
```

```
            "status":  "success",
            "message": "Reading published to Kafka.",
            "data": {
                "reading":   reading,
                "partition": metadata["partition"],
                "offset":    metadata["offset"],
            },
        }), 201

    except Exception as exc:
        app.logger.error("create_reading error: %s", exc)
        return jsonify({
            "status":  "error",
            "message": "Failed to publish reading.",
        }), 500
```

request.get_json(silent=True): parses the request body as JSON. `silent=True` returns `None` instead of raising an exception on malformed JSON.
Validation order matters:

1. Is the body valid JSON? (400 if not)
2. Are required fields present? (400 if not)
3. Are values the correct type? (422 if not)
4. Do values satisfy business rules? (422 if not)

Validate completely before touching any backend system.
Return 201 Created (not 200) for successful POST requests. The distinction tells the client that a new resource was created.

6.6 Step 5 – Error Handlers

```
#         Global error handlers

@app.errorhandler(404)
def not_found(error):
    """Return JSON 404 instead of Flask's default HTML page."""
    return jsonify({
        "status":  "error",
        "code":    404,
        "message": "The requested resource was not found.",
    }), 404

@app.errorhandler(405)
def method_not_allowed(error):
    return jsonify({
        "status":  "error",
        "code":    405,
        "message": "HTTP method not allowed for this endpoint.",
    }), 405
```

```
@app.errorhandler(500)
def internal_error(error):
    return jsonify({
        "status":  "error",
        "code":    500,
        "message": "An internal server error occurred.",
    }), 500

#         App entry point

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

@app.errorhandler(code) registers a function that Flask calls when an unhandled error of that type occurs anywhere in the application. Without these handlers, Flask returns HTML error pages, which break API consumers expecting JSON.
Registering a 404 handler also catches `abort(404)` calls from within route functions, so you only need to define the error format once.

6.7 Step 6 – Test with curl

Open a second terminal. The API server must be running (`python sensor_api/app.py`).

6.7.1 6a – Health Check

```
curl -s http://localhost:5000/api/v1/health | python3 -m
   json.tool
```

-s silences curl's progress bar. **python3 -m json.tool** pretty-prints the JSON response. Expected output:

```
{
  "status":    "ok",
  "version":   "1.0",
  "timestamp": "2024-01-15T10:23:45+00:00",
  "service":   "sensor-data-api"
}
```

6.7.2 6b – List Sensors

```
curl -s http://localhost:5000/api/v1/sensors | python3 -m
   json.tool
```

6.7.3 6c – Latest Reading

```
curl -s \
  http://localhost:5000/api/v1/sensors/temperature/latest \
  | python3 -m json.tool
```

```
# Test invalid sensor type (expect 404)
curl -s \
  http://localhost:5000/api/v1/sensors/radar/latest \
  | python3 -m json.tool
```

6.7.4 6d – Statistics with Query Parameter

```
curl -s \
  "http://localhost:5000/api/v1/sensors/temperature/stats?days=3"
     \
  | python3 -m json.tool

# Test invalid days parameter (expect 400)
curl -s \
  "http://localhost:5000/api/v1/sensors/temperature/stats?days=abc"
     \
  | python3 -m json.tool
```

6.7.5 6e – POST a New Reading

```
curl -s -X POST \
  http://localhost:5000/api/v1/readings \
  -H "Content-Type: application/json" \
  -d '{"sensor":"temperature","value":29.3,"unit":"C"}' \
  | python3 -m json.tool

# Test missing field (expect 400)
curl -s -X POST \
  http://localhost:5000/api/v1/readings \
  -H "Content-Type: application/json" \
  -d '{"sensor":"temperature"}' \
  | python3 -m json.tool

# Test wrong Content-Type (expect 400)
curl -s -X POST \
  http://localhost:5000/api/v1/readings \
  -d 'sensor=temperature&value=29.3' \
  | python3 -m json.tool
```

-X POST: specify the HTTP method. **-H "Content-Type: application/json"**: required header so Flask knows to parse the body as JSON. **-d '{...}'**: the request body.
The third test sends form-encoded data instead of JSON. `request.get_json(silent=True)` returns `None` (Content-Type mismatch), and the API correctly responds with 400.

Reflection Questions

Answer before Session 6:

1. A client sends `GET /api/v1/sensors/temperature/stats` with no `days` parameter. The default is 7. Is it correct to return 200 or 400? Justify your answer.

2. Explain why `POST /readings` is not idempotent but `PUT /readings/42` (replacing reading 42) is. Give a concrete scenario where this distinction matters in a retry mechanism.

3. A colleague suggests returning `200 OK` for all responses and putting the status code in the JSON body (e.g., `{"status": 404, ...}`). What are the problems with this approach?

4. What is the difference between a `400 Bad Request` and a `422 Unprocessable Entity`? Give one example of each for the `POST /readings` endpoint.

5. The `GET /sensors/temperature/latest` endpoint queries Kafka. If the Kafka cluster is down, what HTTP status code should the API return? What should the response body look like?

7. Summary & Key Takeaways

What We Covered Today

1. A **REST API** is the serving layer of the data platform: it exposes processed data to consumers via HTTP.
2. The **six REST constraints** (statelessness, uniform interface, client-server separation, cacheability, layered system, code on demand) define what makes a service RESTful.
3. **HTTP verbs** express intent: GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE. Idempotency and safety have concrete implications for retry logic.
4. **Status codes** must be precise: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 422 Unprocessable Entity, 500.
5. **URL design**: nouns not verbs, plural collections, hierarchical resources, query parameters for filtering.
6. **Flask** provides routing, request parsing, and `jsonify()` in a minimal package. Always register global error handlers to return JSON instead of HTML errors.
7. In the lab, we built a complete REST API serving live Kafka readings and Parquet statistics, with full input validation and structured error responses.

7.1 Vocabulary Checklist

- ☑ REST / RESTful
- ☑ Statelessness
- ☑ Uniform interface
- ☑ Resource / representation
- ☑ URI / URL
- ☑ HTTP verb (GET/POST/PUT/PATCH/DELETE)
- ☑ Idempotent / safe
- ☑ CRUD
- ☑ Status codes (2xx/4xx/5xx)
- ☑ Request / response headers
- ☑ Content-Type / Accept
- ☑ JSON envelope / payload
- ☑ Query parameter vs path parameter
- ☑ API versioning
- ☑ Flask route / view function
- ☑ jsonify / abort
- ☑ Health check endpoint
- ☑ Error handler

7.2 Further Reading

- Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. Chapter 5 (REST). ics.uci.edu/~fielding

- Flask documentation: flask.palletsprojects.com
- HTTP Status Codes reference: developer.mozilla.org/en-US/docs/Web/HTTP/Status
- Richardson Maturity Model (REST maturity levels): martinfowler.com
- Masse, M. (2011). *REST API Design Rulebook*. O'Reilly.

7.3 Preview: Session 6

Next session we harden and extend the API:

- Authentication: API keys and JWT Bearer tokens.
- Rate limiting: protect against abuse.
- OpenAPI / Swagger: auto-generate interactive documentation.
- Lab: Secure the Session 5 API; consume a third-party weather API to enrich sensor data.

Appendix A – Complete API Endpoint Reference

Table 6: All endpoints of the Session 5 Sensor API

lightbg **Verb**	**URL**	**Description**	**Code**
GET	`/api/v1/health`	Health check	200
GET	`/api/v1/sensors`	List all sensor types	200
GET	`/api/v1/sensors/{type}/latest`	Latest Kafka reading for sensor	200/404
GET	`/api/v1/sensors/{type}/stats`	Daily stats from Parquet (`?days=N`)	200/400/404
POST	`/api/v1/readings`	Publish a new reading to Kafka	201/400/422

Appendix B – Glossary

lightbg Term	Definition
abort()	Flask function that immediately raises an HTTP error response.
API	Application Programming Interface: contract for software interaction.
CRUD	Create, Read, Update, Delete – the four basic data operations.
Content-Type	HTTP header declaring the media type of the request/response body.
Flask	Lightweight Python web framework for building APIs.
HATEOAS	Hypermedia As The Engine Of Application State – REST subconstraint.
HTTP verb	Method indicating intended action (GET, POST, PUT, PATCH, DELETE).
Idempotent	Operation that produces the same result when called multiple times.
jsonify()	Flask function that serialises a dict to a JSON HTTP response.
Path parameter	Variable embedded in the URL path (e.g., `/sensors/{type}`).
Query parameter	Key-value pair after `?` in a URL (e.g., `?days=7`).
REST	Representational State Transfer – architectural style for web APIs.
Resource	Any named, addressable concept exposed by an API.
Safe	Operation with no server-side side effects (GET is safe).
Stateless	Each request contains all information needed; no server session state.
Status code	Three-digit HTTP response code indicating success or failure type.
URI	Uniform Resource Identifier – unique address for a resource.